Entities and experiments. Corpus exploration, proto indexing and enrichment workflows

Digital Scholarly Editions "Arcipelago Ceresa"

Named Entities in Digital Editions. Between Structured Databases and Context-Specific Annotation
Open Editions Workshop, University of Zurich
07.03.2026

Levyn Bürki, Data Science Lab, University of Bern
Peter Dängeli, Data Science Lab, University of Bern

Edition workflow: General approach

- Corpus definition
- Image digitisation (Swiss National Library)
- IIIF upload
- Transkribus (raw transcriptions)
  - Generation of IIIF manifests (as sequences of canvasses/images per document)
  - IIIF manifest-based upload to Transkribus
  - Transcription (primarily automated for print/typescripts, manual for manuscripts)
  - Document export (incl. transformation to project data structure)
- Transcription and annotation in oXygen XML editor (with project framework)
- Web app development (SvelteKit, CETEIcean)

Edition workflow: General approach

- Corpus definition
- Image digitisation (Swiss National Library)
- IIIF upload
- Transkribus (raw transcriptions)
- Generation of IIIF manifests (as sequences of canvasses/images per document)
- IIIF manifest-based upload to Transkribus
- Transcription (primarily automated for print/typescripts, manual for manuscripts)
- Document export (incl. transformation to project data structure)
- Transcription and annotation in oXygen XML editor (with project framework)
- Web app development (SvelteKit, CETEIcean)

Named entity detection and identification

Edition workflow: Named entities

Goal: facilitate **manual tagging** by creating an **automatically compiled list** with possible entries (including authority references)

Approach:

- use an LLM to evaluate the raw Transkribus output of each transcribed document and
- detect entities,
- try to link them to one or more authority records, and
- deduplicate the resulting list
- in order to offer the entries in the linking utility in oXygen

We call this a "**proto index**", an imperfect index of entities that likely occur in the corpus.

It is meant to be a shortcut for the editors that saves them manual querying of authority databases.

The actual entity tagging is done by the editors.

State of work: experimental, explorative (but not too far from production-ready)

Demo

Step 1

Feed documents to LLM and ask to detect entities, then identify them using Wikidata knowledge.

Using a slim local helper service that acts like a plug-in the model can call (via the Model Context Protocol, a simple way for tools to talk to each other).

This allows to search (e.g.) Wikidata directly (quickly, in memory) without extra servers or manual wiring.

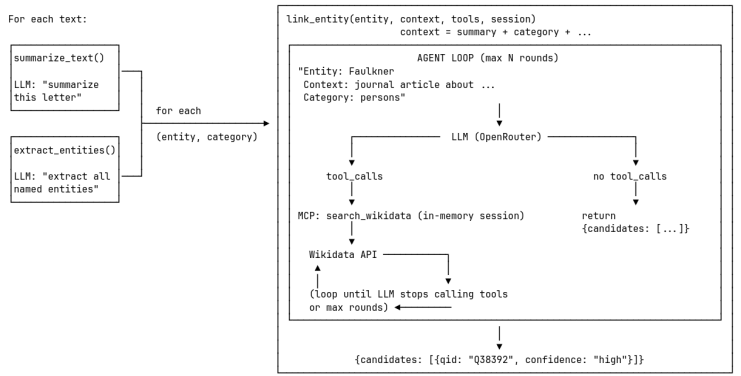
Step 2

Take output of step 1 and enrich it with Wikidata information, then generate a spreadsheet (csv).

Simple Python pipeline executing SPARQL queries (no LLM used).

Demo

Step 1


For each text:
summarize_text()
LLM: "summarize this letter"
extract_entities()
LLM: "extract all named entities"
for each
(entity, category)
link_entity(entity, context, tools, session)
context = summary + category + ...
AGENT LOOP (max N rounds)
"Entity: Faulkner
Context: journal article about ...
Category: persons"
LLM (OpenRouter)
tool_calls
no tool_calls
MCP: search_wikidata (in-memory session)
return
{candidates: [...]}
Wikidata API
(loop until LLM stops calling tools
or max rounds)
{candidates: [{qid: "Q38392", confidence: "high"}]}

Step 1

Feed documents to LLM and ask to detect entities, then identify them using Wikidata knowledge.

```
ENTITY_CLASSES = [
    "persons", "places", "institutions", "publishers", "works", "events", "citations"
]

ENTITY_SEARCH_SYSTEM_PROMPT = f"""
You are a semantic annotation assistant for a Digital Humanities project working with
historical writings (Italian).
Your task: read the text below and extract ALL named entities you can find, grouped by
category.
Return a JSON object with the following top-level keys, each mapping to an array of
exact surface forms as they appear in the text:
{json.dumps({category: [] for category in ENTITY_CLASSES}, indent=2)}
Rules:
- Use the exact string as it appears in the source text (do not normalize or modernize)
- If a category has no entries, return an empty array
"""

ENTITY_LINKING_SYSTEM_PROMPT = """
You are an entity linking assistant for a Digital Humanities project working with
historical writings (Italian).
Your task: link the given entity to a knowledge base (e.g. Wikidata) using the provided
context for disambiguation.
Return a JSON object with a 'candidates' key containing a list of matches, each with a
'qid' and a 'confidence' of high, medium, or low. If no candidates are found, return an
empty list.
"""

TEXT_SUMMARIZATION_SYSTEM_PROMPT = """
historical writings (Italian).
```

```
pip install -r requirements.txt
python main.py > output.txt
```

For executed code see: https://asciinema.org/a/CvUyVktwWlx02NTE?t=43

```
pip install -r requirements.txt
python -m main --input ../../demo/step1/output.txt --output entities_enriched.csv --log
pipeline.log
```

For executed code see: https://asciinema.org/a/CvUyVktwWlx02NTE?t=165

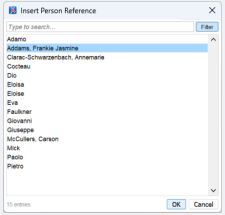
```
csvlens demo/step2/entities_enriched.csv
```

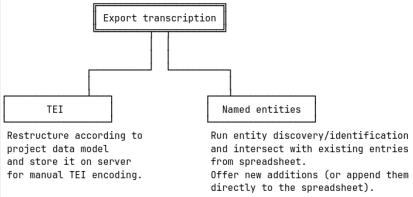
For executed code see: https://asciinema.org/a/CvUyVktwWlx02NTE?t=194

Result: facilitated manual tagging based the automatically compiled list

The generated entries are used to populate the entity spreadsheet(s) of the project (with manual checks).

The oXygen framework queries the spreadsheet and offers the entities for comfortable linking.





Next steps and further considerations

Workflow decisions

We need to define in what frequency and with what degree of automation this recognition/identification task is executed.

One idea is to integrate it into the Transkribus document export. In part, this depends on how fixed the decisions around named entities are (and with them the structure of the spreadsheet).

Technical partner of the Arcipelago Ceresa project

Data Science Lab -- https://dsl.unibe.ch

https://youtu.be/afXUHAUZ4dk